

Task 1: Access and Print the Element at a Given Index in an Array

Aim:

To write a Java program to access and print the element at a given index in an array.

Algorithm:

1. Start.
2. Read the size of the array n .
3. Input the n elements into the array.
4. Read the index i .
5. Check if i is greater than or equal to 0 and less than n .
6. If the index is valid, print $array[i]$.
7. Otherwise, print "Invalid Index".
8. Stop.

Program :

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int index = sc.nextInt();

        System.out.println(arr[index]);
    }
}
```

Result:

To write a Java program to access and print the element at a given index in an array is successfully executed and output is verified.

Task 2: Search for a Given Element in a Sorted Array Using Binary Search

Aim:

To write a Java program to search for a given element in a sorted array using Binary Search.

Algorithm:

1. Start.
2. Read the size of the array `n`.
3. Input the `n` sorted elements into the array.
4. Read the element to be searched (`key`).
5. Initialize:
 - `low = 0`
 - `high = n - 1`
6. Repeat while `low <= high`:
 - Find the middle index: `mid = (low + high) / 2`
 - If `array[mid] == key`, print **"Element Found at index mid"** and stop.
 - If `key < array[mid]`, set `high = mid - 1`.
 - Otherwise, set `low = mid + 1`.
7. If the loop ends without finding the element, print **"Element Not Found"**.
8. Stop.

Program:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int key = sc.nextInt();
        int low = 0, high = n - 1;
        int index = -1;

        while (low <= high) {
            int mid = (low + high) / 2;
```

```
        if (arr[mid] == key) {
            index = mid;
            break;
        } else if (arr[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (index != -1)
        System.out.println(index);
    else
        System.out.println("Element not found");
}
}
```

Result:

To write a Java program to search for a given element in a sorted array using Binary Search is successfully executed and output is verified.

Task 3: Find the Maximum Element in an Array of n Integers

Aim:

To write a Java program to find the maximum element in an array of n integers.

Algorithm:

1. Start.
2. Read the size of the array **n**.
3. Input the **n** elements into the array.
4. Set **max = array[0]**.
5. Repeat for each element from index **1** to **n-1**:
 - If **array[i] > max**, then set **max = array[i]**.
6. Print "**Maximum Element = max**".
7. Stop.

Program:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int max = arr[0];

        for (int i = 1; i < n; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }

        System.out.println(max);
    }
}
```

Result:

Thus To write a Java program to find the maximum element in an array of n integers is successfully executed and output is verified.

Task 4: Find the Kth Smallest Element in an Array

Aim:

To write a Java program to find the Kth smallest element in an array.

Algorithm:

1. Start.
2. Read the size of the array n .
3. Input the n elements into the array.
4. Read the value of k .
5. Sort the array in ascending order.
6. Check if k is greater than or equal to 1 and less than or equal to n .
7. If valid, print the element at index $k - 1$ (the Kth smallest element).
8. Otherwise, print **"Invalid value of K"**.
9. Stop.

Program:

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int k = sc.nextInt();

        Arrays.sort(arr);

        System.out.println(arr[k - 1]);
    }
}
```

Result:

Thus To write a Java program to find the Kth smallest element in an array is successfully executed and output is verified.

Task 5: Print All Possible Pairs of Elements from an Array of Size n

Aim:

To write a Java program to print all possible pairs of elements from an array.

Algorithm:

1. Start.
2. Read the size of the array n .
3. Input the n elements into the array.
4. For $i = 0$ to $n - 2$:
 - For $j = i + 1$ to $n - 1$:
 - Print the pair ($array[i]$, $array[j]$).
5. Stop.

Program:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                System.out.println(arr[i] + " " + arr[j]);
            }
        }
    }
}
```

Result:

Thus To write a Java program to print all possible pairs of elements from an array. is successfully executed and output is verified.